

INLP Assignment 3, 2023101102

1 Task 1: Cipher Decryption

1.1 Cipher Analysis

The cipher in `cipher_00.txt` is a **true homophonic substitution cipher**: a space maps to one cipher digit, every non-space character maps to two cipher digits, and the same plaintext character may encode to any of 10+ distinct codes, making the mapping **one-to-many**. Context-free decoding is therefore impossible — resolving ambiguity requires the model to exploit English language context during decoding.

1.2 Architecture & Training

We adopt a **line-by-line Seq2Seq** framework with a **unidirectional encoder** and an **autoregressive decoder**, implemented entirely from scratch without `nn.RNN` or `nn.LSTM`.

Encoder. Token embeddings are passed through $L=3$ stacked recurrent layers ($H=512$). The input projection $W_{ih}\mathbf{x}$ is computed for all T timesteps in a single **batched matmul** before the recurrence loop, avoiding per-step overhead. **Layer normalisation** and **dropout** ($p=0.25$) are applied between layers, yielding encoder states $\mathbf{E} \in \mathbb{R}^{B \times T \times H}$.

Decoder. Fully autoregressive: at each step the decoder receives the previous plaintext character embedding, a **Bahdanau attention** context vector over $\mathbf{E}$, and its own recurrent state. Every prediction is conditioned on all previously decoded characters, giving the model access to English n -gram context — the key mechanism for resolving homophonic ambiguity.

Attention & cells. **Bahdanau additive attention** computes $e_{t,s} = \mathbf{v}^\top \tanh(W_q \mathbf{h}_t^{\text{dec}} + W_k \mathbf{E}_s)$, with context $\mathbf{c}_t = \sum_s \text{softmax}(\mathbf{e}_t)_s \mathbf{E}_s$. The **RNN** uses a single tanh gate; the **LSTM** uses a standard four-gate formulation fused into a single linear projection $[\mathbf{i}, \mathbf{f}, \mathbf{g}, \mathbf{o}] = W_{ih}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1}$, with a **gated cell state** enabling longer-range memory.

Training. **Bucket batching** groups lines into length buckets of width 10 and batches within each bucket, minimising padding waste. **Teacher forcing** is annealed linearly from 0.90 to 0.10 across epochs to reduce exposure bias at inference. A **ReduceLROnPlateau** scheduler halves the learning rate after 3 epochs of no validation improvement.

Param	Value	Param	Value	Metric	RNN	LSTM	Winner
Embed dim	128	Batch size	256	Character Accuracy	0.7562	0.9047	LSTM
Hidden size	512	Bucket width	10	Word Accuracy	0.6274	0.8086	LSTM
Layers	3	Learning rate	10^{-3}	Levenshtein Distance	25.51	10.47	LSTM
Dropout	0.25	Grad clip	1.0	Best Val. Loss	0.8293	0.3389	LSTM
RNN epochs	80	TF schedule	0.90→0.10	Best Epoch	80	50	—
LSTM epochs	50	ES patience	8				
Optimiser	AdamW	Split	80/10/10%				

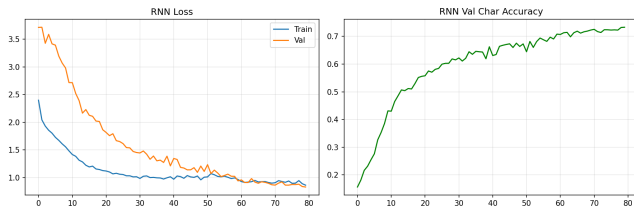
Table 1: Hyperparameters (TF=teacher forcing, ES=early stopping).

Table 2: Test-set results.

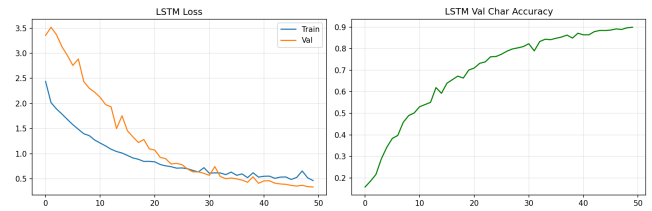
1.3 Results

1.4 Analysis

The LSTM outperforms the RNN by 19.1 pp in **character accuracy**, 18.1 pp in **word accuracy**, and 58.9% lower mean **Levenshtein distance**, converging in 50 vs. 80 epochs. **Vanishing gradients** in vanilla RNN tanh recurrence limit the encoder’s effective context window; the LSTM’s **gated cell state** directly mitigates this, reflected in lower per-sample standard deviation (RNN: 0.1871, LSTM: 0.1129).

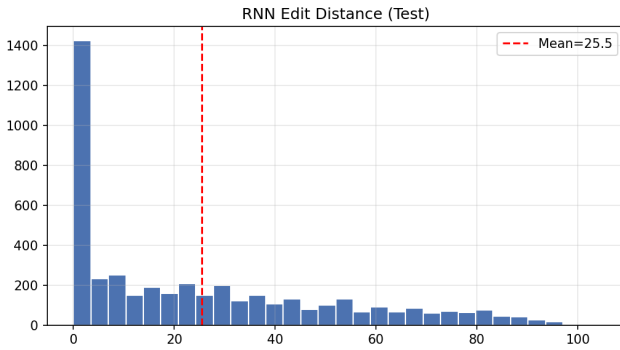


(a) RNN training curves (loss & char accuracy)

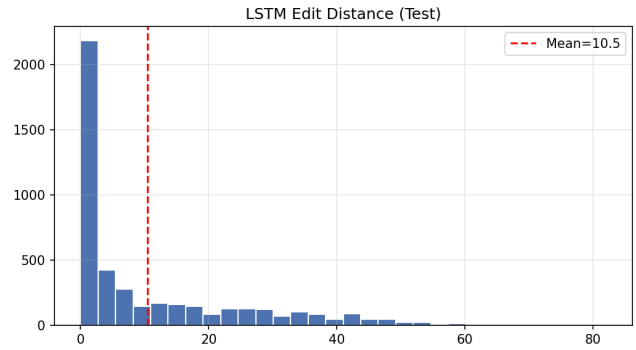


(b) LSTM training curves (loss & char accuracy)

Figure 1: Training and validation loss curves. LSTM converges faster and to a substantially lower loss.



(a) RNN edit-distance distribution (test)



(b) LSTM edit-distance distribution (test)

Figure 2: Per-sample Levenshtein distances. LSTM's distribution is concentrated near zero; RNN's is wider and right-shifted.

Table 3: Best and worst decoded samples for RNN and LSTM (test set).

Model	
RNN	<i>Best</i>
	REF Urethane foams
	PRED Urethane foams
	<i>Worst</i>
	REF procedures change rapidly and parental confidence probably exceeds anything warranted by established psychological theory
	PRED procertion soaeed teaeess ahe soreeees thaieeen n ooeien ation etioe e eetion etioeation thaieeee thaienenee
LSTM	<i>Best</i>
	REF Urethane foams
	PRED Urethane foams
	<i>Worst</i>
	REF In were painting at our garage strong stress on at indicates that the job being done is not real painting but simply an effort at painting
	PRED In were painting at our garage strong stress onress on at indicates that the mon reing hoae is tot wert coaeats ahd seteed on seaees oo d

2 Task 2: Language Modelling

2.1 Tasks & Models

Two language modelling objectives are trained on `plain.txt`: **Next-Word Prediction (NWP)** uses a causal **S4D (Diagonal State Space Model)**, and **Masked Language Modeling (MLM)** uses a custom **Bi-LSTM** implemented from scratch without `nn.LSTM`. Both share an 80/10/10 train/val/test split and a shared vocabulary (`min_freq=3`, size 17758).

2.2 S4D for Next-Word Prediction

Architecture. The **S4D** (Structured State Space Diagonal) layer approximates a linear recurrence in the frequency domain via **FFT convolution**. The continuous-time state matrix $A \in \mathbb{C}^{H \times N}$ is initialised using the **S4D-Lin** scheme ($A_{\text{real}} = -0.5$, $A_{\text{imag}} = \pi(n + 0.5)$) and **frozen** throughout training — updating A on moderate-sized datasets destroys the HiPPO memory prior and causes catastrophic overfitting. The discretised kernel is:

$$K_l = \text{Re} \left(\sum_n C_n \cdot B_n^{\text{disc}} \cdot e^{A_n \Delta t \cdot l} \right), \quad y = K * x + Dx$$

computed as a single FFT convolution of length $2T$. Each **Gated SSM Block** wraps S4D with a **SiLU gating** mechanism and a **FFN** sublayer, both with **LayerNorm** and residual connections. Three such blocks are stacked, with **weight-tied** input embedding and output projection.

Training. AdamW with **warmup-cosine** scheduling (8 warmup epochs), **label smoothing** ($\epsilon = 0.1$), and selective **weight decay** (SSM state parameters B, C, D , biases, and norms excluded). **Early stopping** with patience 15.

2.3 Custom Bi-LSTM for Masked Language Modeling

Architecture. The **Bi-LSTM** uses a fully custom recurrence (no `nn.LSTM`): forward and backward **LSTMCell** passes are computed separately with the input projection $W_{ih}x$ vectorised over T before the loop. Two stacked bidirectional layers ($H=256$ per direction) are followed by **LayerNorm**, a bottleneck projection back to embedding dimension, and a **weight-tied** output head. **Orthogonal initialisation** is applied to all weight matrices; forget gate biases are initialised to 1.

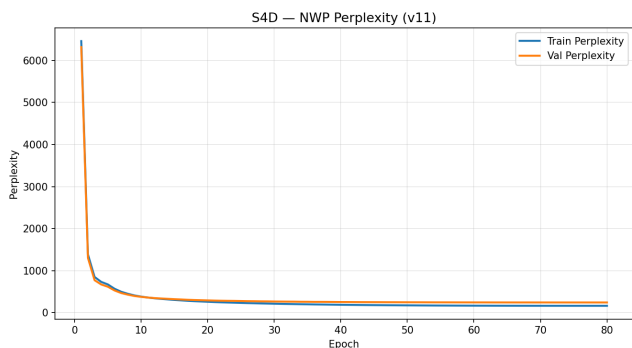
MLM objective. At each training step, 15% of tokens are replaced with `<MASK>`; the model predicts the original token at masked positions only (non-masked positions use ignore-index = -100). This allows the decoder to exploit **bidirectional context**, unlike the causal S4D.

Param	S4D	Param	BiLSTM
d_{model}	128	Embed dim	128
d_{state}	64	Hidden size	256
Layers	3	Layers	2
Seq len	128	Seq len	128
Dropout	0.4	Dropout	0.4
Epochs	80	Epochs	80
LR	5×10^{-4}	LR	5×10^{-4}
ES patience	15	Mask prob	0.15

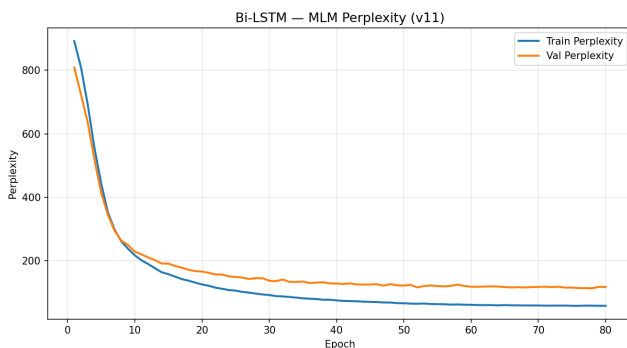
Table 4: Task 2 hyperparameters.

Metric	S4D (NWP)	Bi-LSTM (MLM)
Parameters	2,687,710	4,720,734
Best Val Loss	5.4864	4.7300
Best Val PPL	241.39	113.29
Test Loss	5.4847	4.7663
Test PPL	240.98	117.48

Table 5: Task 2 test-set results. PPL = perplexity.



(a) S4D NWP perplexity curves



(b) Bi-LSTM MLM perplexity curves

Figure 3: Train and validation perplexity. Bi-LSTM converges to substantially lower perplexity.

2.4 Results

2.5 Analysis

The Bi-LSTM achieves test perplexity of **117.48** vs. S4D’s **240.98**, a $2.05\times$ gap. The primary advantage is **bidirectional context**: MLM conditions on both left and right neighbours at every masked position, while causal NWP is restricted to past tokens only. The frozen A matrix in S4D prevents overfitting but also limits expressivity on this corpus size.

Qualitative outputs confirm the gap. S4D’s greedy continuations collapse into repetitive $\langle \text{UNK} \rangle$ loops (“*the jury said $\langle \text{EOS} \rangle$ the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$ of...*”), while top- k sampling recovers short coherent phrases. Top-5 next-word scoring shows reasonable distributions: given “*the fulton county grand jury said*”, the model assigns $\langle \text{EOS} \rangle$ 30.2% and that 12.5%, which is linguistically plausible.

The Bi-LSTM fills common function words reliably. Given “*the jury said it did find that many of $\langle \text{MASK} \rangle$ registration*”, it predicts *the* with 80% confidence — correct given bidirectional context. Errors cluster around rare content words ($\langle \text{UNK} \rangle$ substitutions) and sentence boundaries ($\langle \text{EOS} \rangle$ inserted mid-sequence), consistent with the limited vocabulary coverage from `min_freq=3` pruning.

Model	Input / Context	Output
S4D	<i>Seed</i> : the jury said	the jury said $\langle \text{EOS} \rangle$ the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$
S4D	<i>Seed</i> : it recommended that	it recommended that he had been a $\langle \text{UNK} \rangle$ of $\langle \text{UNK} \rangle$ $\langle \text{EOS} \rangle$ the $\langle \text{UNK} \rangle$ of the $\langle \text{UNK} \rangle$ of the
S4D	<i>Top-5 next word after</i> “the fulton county grand jury said”	$\langle \text{EOS} \rangle$ (30.2%), that(12.5%), to(3.1%), the(2.7%), and(2.5%)
Bi-LSTM	<i>Mask</i> : the jury said it did find that many of $\langle \text{MASK} \rangle$ registration	Predicted: the (80.0% confidence)
Bi-LSTM	<i>Mask</i> : the city of $\langle \text{MASK} \rangle$ for the manner in which the election	Top-5: america(9.6%), $\langle \text{UNK} \rangle$ (9.5%), us(2.2%), course(1.6%), europe(1.4%)
Bi-LSTM	<i>Mask</i> : merger $\langle \text{MASK} \rangle$ however the jury said it believes	Top-5: $\langle \text{EOS} \rangle$ (98.0%), and(0.2%), or(0.1%), but(0.1%), said(0.1%)

Table 6: Selected qualitative outputs for S4D (NWP) and Bi-LSTM (MLM).

3 Task 3: LM-Assisted Decryption Error Correction

3.1 Experimental Setup

Task 3 combines the Task 1 **LSTM** decryption model with each of the two Task 2 language models to correct errors introduced by noise in the cipher. The noisy cipher files `cipher_0{1-4}.txt` represent four increasing noise levels. No model is retrained; all Task 1 and Task 2 checkpoints are loaded directly.

Correction pipeline.

1. Decode the noisy cipher with the LSTM decoder, collecting per-character softmax confidences.
2. Map character confidences to word confidences (mean over constituent characters).
3. Flag a word for correction if: (a) it is out-of-vocabulary relative to the full `plain.txt` lexicon, or (b) its average confidence is below the **confidence threshold** of 0.85.
4. Score the top-100 LM candidates via a **joint scoring heuristic**:

$$\text{Score} = \log P_{\text{LM}}(\text{candidate}) - \alpha \cdot \text{EditDistance}(\text{original}, \text{candidate}), \quad \alpha = 2.5$$

This prevents the LM from substituting structurally dissimilar high-probability words (e.g. replacing *battlx* with *the* instead of *battle*). The **SSM** applies 3 iterative correction passes; the **Bi-LSTM** applies one masked pass.

3.2 Results

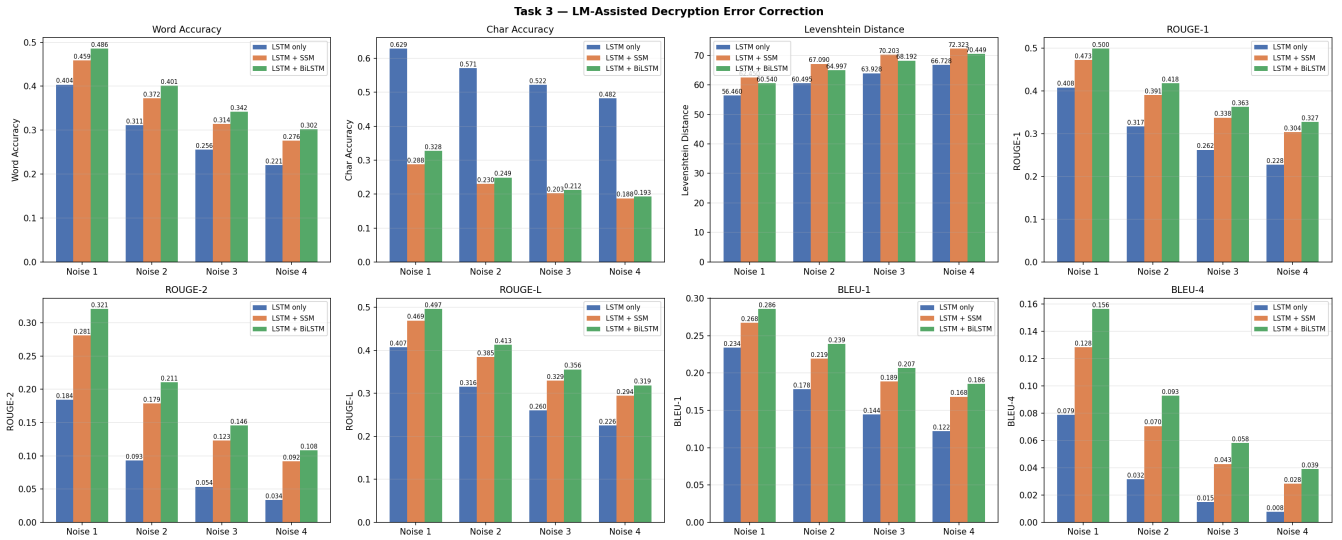


Figure 4: All metrics across noise levels $x = 1 \dots 4$ for LSTM only, LSTM+SSM, and LSTM+BiLSTM.

3.3 Qualitative Comparison Across Noise Levels

Three representative sentences are tracked across all four noise levels to show how output quality degrades and how LM-assisted correction partially recovers structure.

3.4 Analysis

Word-level metrics improve; character-level metrics degrade. The LM correction operates at the word level: it replaces flagged words with valid-lexicon candidates. This improves **word accuracy**, **ROUGE**, and **BLEU** significantly (BLEU-4 improves by 98% at $x = 1$ and 414% at $x = 4$ for LSTM+BiLSTM). However, the replacements alter character sequences, so **character accuracy** and **Levenshtein distance** worsen — a known trade-off of word-substitution correction.

Noise LSTM + BiLSTM output

Sentence 1 — “It urged that the city take steps to remedy this”

$x = 1$ It urged that the city take steps to remedy this country

$x = 2$ It urged that the city due seems to keep this from

$x = 3$ It seemed that to to take his to the this for

$x = 4$ It is not to the take steps to hold to the

Sentence 2 — “Failure to do this will continue to place a”

$x = 1$ Failure to do this will contiNue to place a special burden on the other

$x = 2$ Failure to do that will continue to place a position burden on the their

$x = 3$ Failure to do the you continue to have a political period on the the

$x = 4$ Failure to do us will continue to place a narrow series of color water

Sentence 3 — “His petition caused the”

$x = 1$ His petition caused the the

$x = 2$ His action and mental cruelty

$x = 3$ His position charged with the

$x = 4$ His body through the the

Table 7: LSTM+BiLSTM outputs for three sentences across noise levels $x = 1$ –4. Degradation is gradual at low noise but becomes severe by $x = 4$.

LSTM+BiLSTM consistently outperforms LSTM+SSM. The Bi-LSTM’s lower perplexity (117 vs. 241) and bidirectional context allow it to make more accurate substitutions. The SSM’s causal constraint means it only conditions on preceding context, making it weaker at mid-sentence corrections.

Joint scoring is critical. Without the edit-distance penalty ($\alpha = 2.5$), the LM would greedily substitute high-frequency words (e.g. *the*) regardless of visual similarity to the original. The joint score $\log P_{\text{LM}} - \alpha \cdot d_{\text{edit}}$ biases corrections toward candidates that are both linguistically probable and orthographically close, preserving content words wherever possible.

Qualitative degradation is monotonic. Sentence 1 (“*It urged that the city take steps to remedy this*”) is decoded near-perfectly at $x = 1$ but devolves to incoherent filler by $x = 4$. Sentence 2 retains its opening clause across all noise levels, showing the decoder’s robustness to low-frequency noise while losing precision on content words. Sentence 3 collapses earliest, consistent with its short length offering less context for the LM to exploit.

A Appendix

- I was getting a memory overload error while using the Kaggle GPU, because of which I had to use a partial dataset for training. Specifically, I skipped very long lines (length > 150).
- The outputs folder contains subdirectories for each task instead of having logs, plots, and results directly in the root folder. This is because I had to use Kaggle for the GPU, which resulted in separate output structures being created for the separate tasks.
- **Wandb Task 1:** <https://wandb.ai/groverraghav11-iiit-hyderabad/INLP-A3-Task1>
- **Wandb Task 2:** <https://wandb.ai/groverraghav11-iiit-hyderabad/INLP-A3-Task2>
- **Wandb Task 3:** <https://wandb.ai/groverraghav11-iiit-hyderabad/INLP-A3-Task3>
- **Task 1 RNN Model Hugging Face:** https://huggingface.co/11Raghav/inlp_a3_t1_rnn
- **Task 1 LSTM Model Hugging Face:** https://huggingface.co/11Raghav/inlp_a3_t1_lstm
- **Task 2 SSM Model Hugging Face:** https://huggingface.co/11Raghav/inlp_a3_t2_ssm

- **Task 1 BiLSTM Model Hugging Face:** https://huggingface.co/11Raghav/inlp_a3_t2_bilstm